Arquitectura de Integración para Modernización Bancaria

✅ **Documento preparado por**: Oscar Macias
✅ **Documento dirigido a**: Devsu
✅ **Vigencia**: Se establece duración estimada

1️⃣ Executive Summary

📌 Declaración Ejecutiva

Este documento presenta una propuesta de arquitectura de integración para la modernización tecnológica de un banco tradicional que busca mejorar sus capacidades digitales y adaptarse a nuevas tendencias regulatorias y de industria como Open Finance.

La solución propuesta permite la coexistencia entre un **core bancario legacy y un nuevo core digital**, mediante una arquitectura desacoplada basada en **APIs y eventos**, que facilita la evolución progresiva del ecosistema tecnológico.

La arquitectura propuesta garantiza:

✅ *Integración multicore sin impacto en canales*
✅ *Arquitectura orientada a eventos*
✅ *Gobierno formal de APIs*
✅ *Seguridad por diseño*
✅ *Cumplimiento regulatorio y protección de datos*
✅ *Alta disponibilidad y resiliencia operacional*
✅ *Migración progresiva mediante **Strangler Pattern***

El diseño conceptual se alinea con el estándar **BIAN (Banking Industry Architecture Network)** para organizar capacidades de negocio en dominios funcionales desacoplados.

⭐ Control Documental

Versión	Fecha	Descripción del cambio
1.0	06-mar-26	Emisión inicial

2️⃣ Contexto y Drivers de Modernización

📌 **2.1 El banco busca modernizar su infraestructura tecnológica para responder a nuevas demandas de negocio, regulaciones financieras y competencia digital.**

Los principales objetivos son:

✅ Modernizar los canales digitales
✅ Integrar múltiples sistemas bancarios
✅ Habilitar Open Finance
✅ Reducir dependencia del core legacy
✅ Mejorar resiliencia operacional
✅ Garantizar seguridad y cumplimiento regulatorio

La arquitectura debe integrar los siguientes sistemas:

✅ Core bancario tradicional
✅ Nuevo core bancario digital
✅ Canales de banca web y móvil
✅ Plataforma de pagos
✅ APIs para terceros
✅ Sistema de gestión de riesgos
✅ Sistema de prevención de fraude

Principio arquitectónico clave: **Desacoplar los canales del core bancario.**

3 Arquitectura de Integración – Modelo C4

📌 *Para mayor detalle en calidad, ver documentos png y draw.io.*

La arquitectura se documenta utilizando el modelo **C4**, el cual permite describir sistemas complejos en distintos niveles de abstracción.

Se incluyen los siguientes diagramas:

📌 C4 Context Diagram

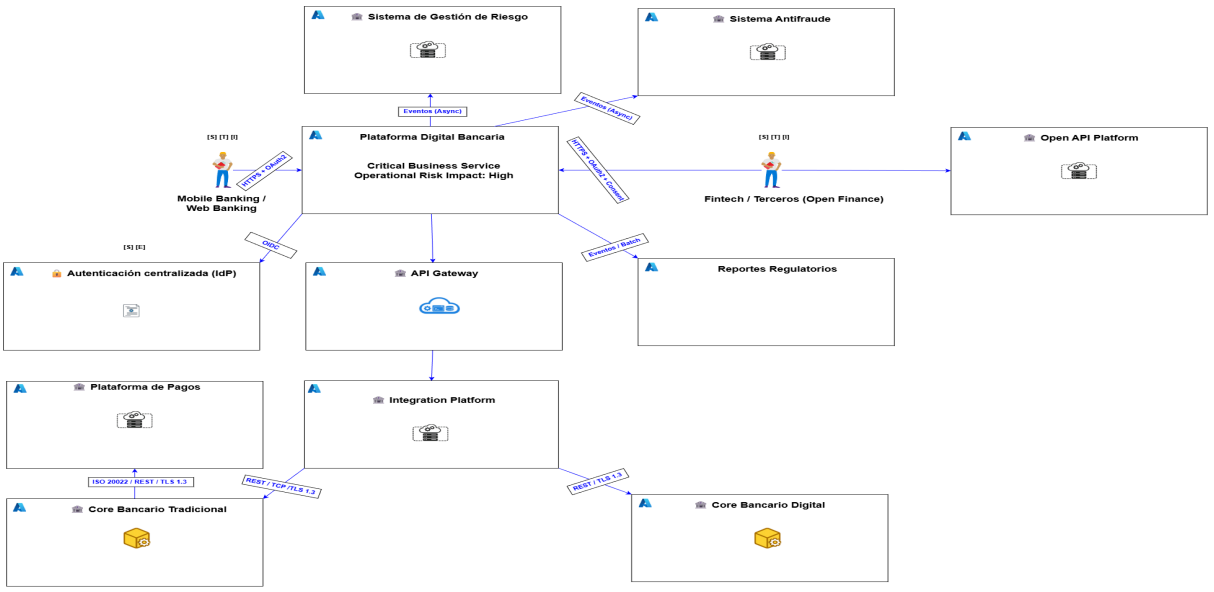
Describe el sistema bancario dentro de su entorno externo.

Actores y sistemas externos:

✅ Clientes del banco
✅ Aplicaciones móviles y web
✅ Fintechs externas

✅ Redes de pago
✅ Sistemas regulatorios

El sistema central es la **Plataforma de Integración Bancaria**, que actúa como capa de desacoplamiento entre canales y sistemas backend.



Context Architecture

El diagrama de contexto presenta la Plataforma de Integración Bancaria como núcleo de desacoplamiento entre canales digitales, terceros bajo esquema Open Finance y los sistemas core del banco.

La arquitectura establece un punto único de entrada seguro mediante APIs gobernadas, garantizando que los canales no dependan directamente de los cores bancarios. La interacción con sistemas externos (core legacy, core digital, pagos, riesgo y fraude) se realiza bajo contratos definidos y mecanismos de integración controlados.

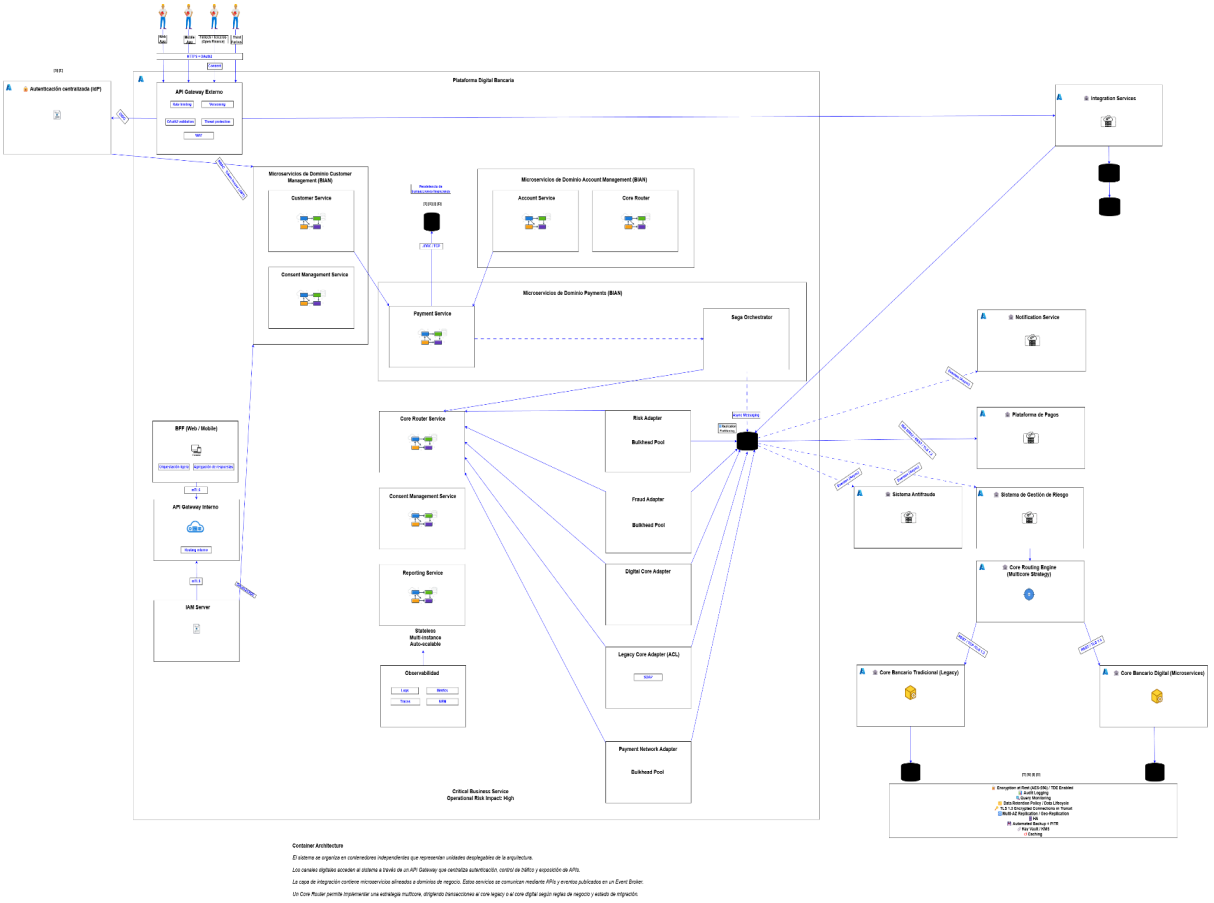
Se evidencia la separación clara entre actores externos, sistemas corporativos y la capa de integración, alineada a principios de desacoplamiento, seguridad por diseño y resiliencia operacional.

📌C4 Container Diagram

Describe los principales contenedores de la arquitectura:

- ✅API Gateway
- ✅Microservicios de negocio
- ✅Event Broker
- ✅Core Router
- ✅Sistemas core bancarios
- ✅Plataforma de pagos
- ✅Sistemas de riesgo y fraude
- ✅Plataforma IAM

Cada contenedor tiene responsabilidades específicas y se comunica mediante APIs o eventos.

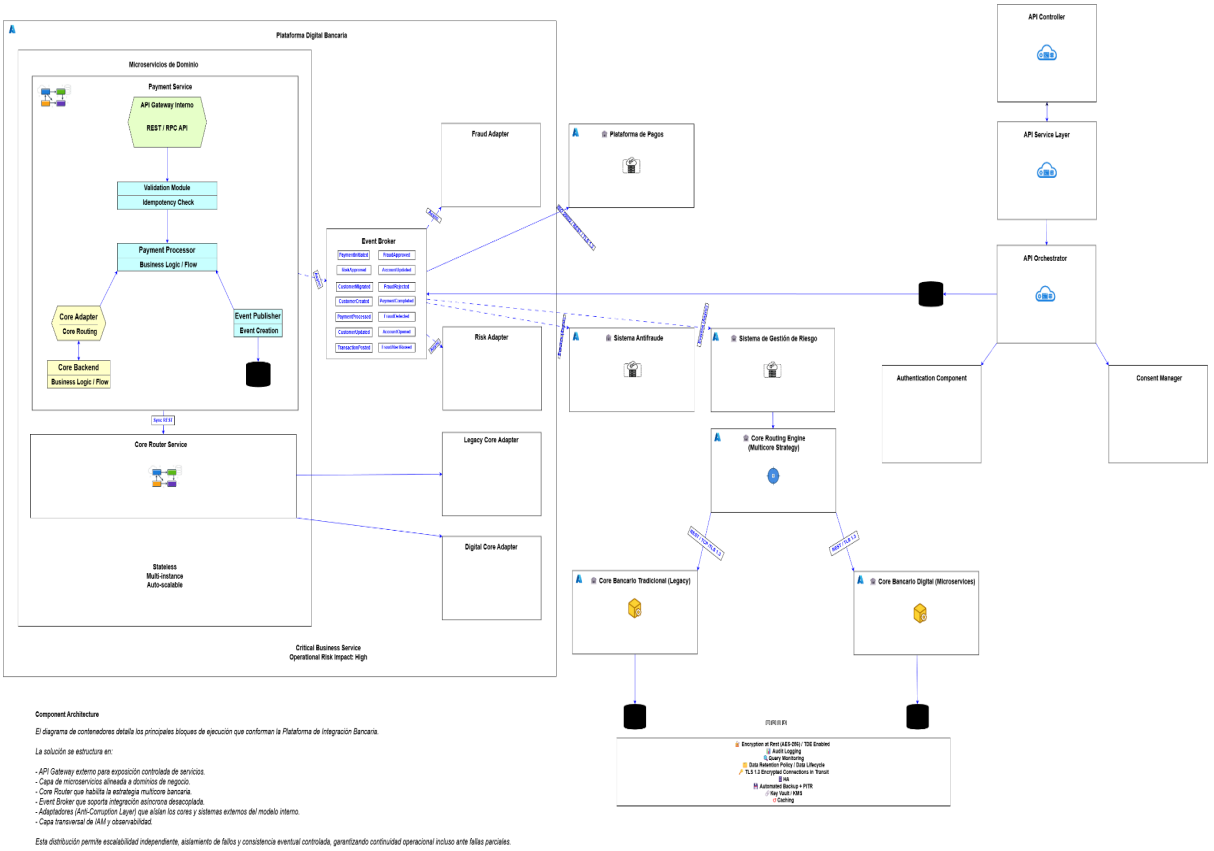


📌C4 Component Diagram

Describe los componentes internos de la capa de integración:

- ✅API Gateway
- ✅Core Router
- ✅Customer Service
- ✅Payments Service
- ✅Fraud Integration Service
- ✅Risk Integration Service
- ✅Event Publisher
- ✅Event Consumers

Los microservicios se organizan por dominio de negocio siguiendo principios **Domain Driven Design**.



4️⃣ Estrategia Multicore Bancaria

La arquitectura soporta la coexistencia entre múltiples cores bancarios.

📌4.1 Objetivo

Permitir migración progresiva desde el core legacy hacia el core digital sin afectar canales ni operaciones.

Componentes clave:

Core Router

Servicio responsable de enrutar solicitudes hacia el core correspondiente.

El enrutamiento se basa en:

✅Tipo de producto
✅Segmento de cliente
✅Estado de migración
✅Tipo de operación

Golden Record de Cliente

Se establece un registro maestro único para evitar duplicidad de información de clientes entre sistemas.

Sincronización por Eventos

Los cambios en los sistemas se sincronizan mediante eventos publicados en el Event Broker.

Ejemplos de eventos:

✅CustomerCreated
✅AccountUpdated
✅TransactionProcessed

Esto permite consistencia eventual entre sistemas.

Migración progresiva

Se utiliza el patrón: **Strangler Pattern**

Permitiendo que nuevos servicios sustituyan gradualmente funcionalidades del sistema legacy.

5️⃣ Arquitectura Orientada a Eventos

📌5.1 Se implementa una Event Driven Architecture (EDA) para desacoplar sistemas y mejorar resiliencia.

Un Event Broker (Apache Kafka) permite la comunicación asincrónica entre servicios.

Casos de uso de eventos:

✅Notificaciones
✅Integración con antifraude
✅Sincronización multicore
✅Auditoría
✅Reportes regulatorios

Beneficios:

✅desacoplamiento
✅escalabilidad
✅resiliencia
✅integración flexible

6️⃣ Estrategia de APIs

La arquitectura se basa en un modelo **API First**.

📌 6.1 APIs externas (Open Finance)

Expuestas a terceros mediante un API Gateway.

Características:

- ✅ OAuth2 Authorization Code + PKCE
- ✅ OpenID Connect
- ✅ Rate limiting
- ✅ Versionamiento semántico
- ✅ Documentación OpenAPI
- ✅ Consentimiento explícito del cliente

📌 6.2 APIs Internas

Utilizadas para comunicación entre microservicios.

Tecnologías posibles:

- ✅ REST
- ✅ gRPC

Seguridad:

- ✅ mTLS
- ✅ JWT internos

Estas APIs no son expuestas públicamente.

7️⃣ Gestión de Identidad y Acceso (IAM)

Se implementa un modelo centralizado de identidad.

Componentes:

✅ Identity Provider
✅ Authorization Server
✅ Token Service

Mecanismos utilizados:

✅ OAuth2
✅ OpenID Connect
✅ JWT firmados
✅ mTLS entre servicios
✅ RBAC

Se aplica el principio de **mínimo privilegio**.

8️⃣ Seguridad y Cumplimiento

La arquitectura considera requisitos regulatorios de protección de datos.

📌 8.1 Alineado (LOPDP, GLBA, CCPA/CPRA):

Medidas de seguridad:

✅ cifrado en tránsito (TLS 1.3)
✅ cifrado en reposo (AES-256)
✅ tokenización de datos sensibles
✅ control de acceso basado en roles
✅ auditoría completa
✅ logs inmutables
✅ monitoreo mediante SIEM

El diseño considera **modelado de amenazas basado en STRIDE**.

9️⃣ Alta Disponibilidad y Recuperación ante Desastres

📌La arquitectura se diseña para resiliencia operacional.

Componentes críticos:

✅API Gateway
✅Event Broker
✅Core Router
✅IAM
✅Bases de datos transaccionales

Estrategias implementadas:

✅despliegue multi-zona
✅microservicios active-active
✅replicación de bases de datos
✅circuit breaker
✅retry policies
✅health checks
✅monitoreo continuo

Objetivos operativos:

✅**RTO**: menor a 1 hora
✅**RPO**: menor a 5 minutos

⭐ Gobierno de APIs y Microservicios

Se establece un modelo de gobierno para garantizar consistencia y control.

Elementos del modelo:

✅diseño contract-first
✅catálogo central de APIs
✅versionamiento controlado
✅revisiones de arquitectura
✅observabilidad obligatoria
✅CI/CD con controles de seguridad
✅ownership por dominio

Esto permite una evolución controlada de la arquitectura.

⭐ Plan de Migración Gradual

La modernización se ejecuta en fases para reducir riesgo operativo.

📌*Fase 1*:

✅Encapsular el core legacy mediante una capa de APIs.

📌*Fase 2*:

✅Implementar el nuevo core digital para nuevos clientes.

📌*Fase 3*:

✅Introducir el Core Router.

📌*Fase 4*:

✅Implementar sincronización por eventos.

📌*Fase 5*:

✅Migración progresiva de clientes y productos.

📌*Fase 6*:

✅Decommission gradual del core legacy.

⭐ Alineación Conceptual a BIAN

La arquitectura se alinea conceptualmente con BIAN.

Los microservicios representan dominios funcionales bancarios como:

✅Customer Management
✅Account Management
✅Payments
✅Fraud Management
✅Risk Management

Esto permite:

✅separación clara de capacidades
✅evolución independiente
✅interoperabilidad entre sistemas

🔟Conclusión

📌La arquitectura propuesta permite modernizar el ecosistema tecnológico del banco mediante una plataforma de integración desacoplada basada en APIs y eventos.

Los principales beneficios incluyen:

✅integración multicore
✅soporte para Open Finance
✅resiliencia operacional
✅escalabilidad
✅seguridad y cumplimiento regulatorio
✅migración progresiva controlada

Esta arquitectura permite al banco evolucionar hacia un modelo digital sin comprometer la estabilidad de sus sistemas actuales.